Databases 2 - exam - September 12, 2023 - Dur. 2h

S. Comai, P. Fraternali, D. Martinenghi

A. Concurrency Control (7 points)

Classify the following schedule S with respect to VSR, CSR, 2PL, Strict 2PL, TS-mono, and TS-multi. Motivate all your answers.

- If it belongs to VSR, provide all the possible serializations.
- If it does not belong to 2PL and/or 2PL-strict, explain which lock/unlock requests cause this.

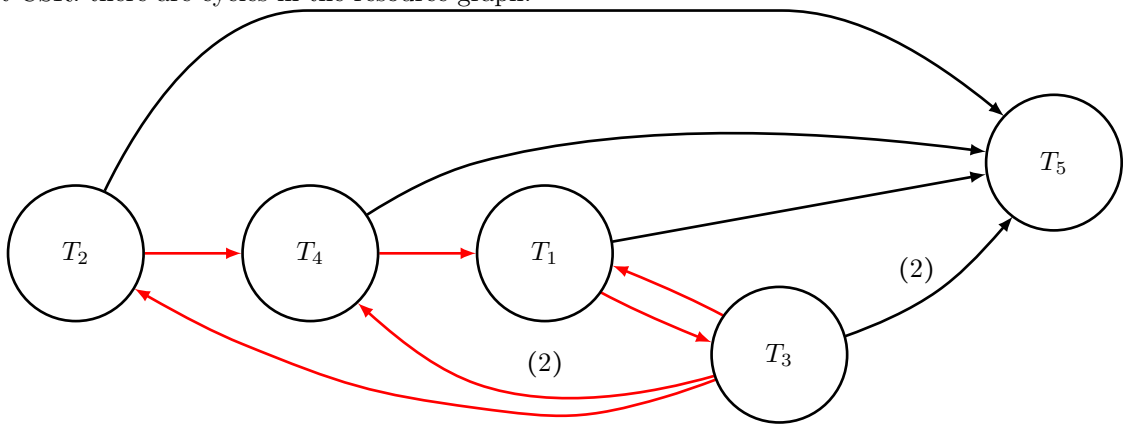
$$S = r_3(X)\ w_3(Y)\ w_2(Y)\ w_1(Z)\ r_3(Z)\ w_4(X)\ r_4(Y)\ w_1(X)\ r_5(Y)\ w_5(X)$$

If additional space is required, please use the last page.

Solution.

Detailed analysis for the CSR class

Not CSR: there are cycles in the resource graph.



Detailed analysis for the VSR class

Not VSR: the schedule is not CSR and no permutation/relocation of its blind writes $\{ w_4(X), w_1(X), w_3(Y) \}$ makes it CSR.

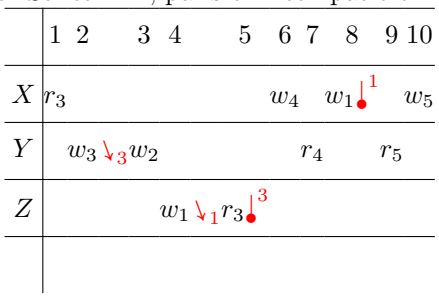
Detailed analysis for the 2PL and Strict 2PL classes

Not 2PL or Strict 2PL as S is not CSR. In particular, $w_1(X)$ cannot anticipate the lock before $w_4(X)$ and $w_1(Z)$ must release the lock before $r_3(Z)$. Similarly, $r_3(Z)$ cannot anticipate the lock request and $w_3(Y)$ must release the lock before $w_2(Y)$. The table below shows a pair of incompatible requests (circled in red).

	1 2	3 4	5 6 7	8 9	10
X	$\uparrow_3 r_3$		$\downarrow_3 \uparrow_4 w_4 \downarrow_4$ **$\uparrow_1$** $w_1 \downarrow_1 \uparrow_5 w_5 \downarrow_5$		
Y	$\uparrow_3 w_3 \downarrow_3 \uparrow_2 w_2 \downarrow_2$		$\uparrow_4 r_4 \downarrow_4 \uparrow_5 r_5$		$\downarrow_5$
Z		$\uparrow_1 w_1$ **$\downarrow_1$** $\uparrow_3 r_3 \downarrow_3$			
		2	4		5

Legend
$\uparrow$: read lock request
$\uparrow$: write lock request
$\downarrow$: unlock request

For Strict 2PL, pairs of incompatible mandatory unlock requests and commit times are shown below.



Legend
↘: mandatory unlock request
↓: commit time

Detailed analysis for the TS Mono class

Not TS-mono, since $w_1(X)$ occurs after $w_4(X)$ and $w_3(Y)$ after $w_2(Y)$.

Operation	RTM(X)	WTM(X)	RTM(Y)	WTM(Y)	RTM(Z)	WTM(Z)	Killed transactions
$r_3(X)$	3 (0)	0	0 (0)	0	0 (0)	0	
$w_3(Y)$				3			
$w_2(Y)$							2 as 2 < WTM(Y) = 3
$w_1(Z)$						1	
$r_3(Z)$					3 (0)		
$w_4(X)$		4					
$r_4(Y)$			4 (0)				
$w_1(X)$							1 as 1 < WTM(X) = 4
$r_5(Y)$			5 (0)				
$w_5(X)$		5					

Detailed analysis for the TS Multi

Not TS-multi (according to the rule used in practice): still $w_1(X)$ after $w_4(X)$ and $w_3(Y)$ after $w_2(Y)$

Operation	RTM(X)	WTM(X)	RTM(Y)	WTM(Y)	RTM(Z)	WTM(Z)	Killed transactions
$r_3(X)$	3 (0)	0	0 (0)	0	0 (0)	0	
$w_3(Y)$				0, 3			
$w_2(Y)$							2 as 2 < last WTM(Y) = 3
$w_1(Z)$						0, 1	
$r_3(Z)$					3 (1)		
$w_4(X)$		0, 4					
$r_4(Y)$			4 (3)				
$w_1(X)$							1 as 1 < RTM(X) = 3
$r_5(Y)$			5 (3)				
$w_5(X)$		0, 4, 5					

B. Physical Databases (9 points)
`Customer(CUSTID, Name, Email, Phone,...)` has 40K tuples stored in a primary B+ on CUSTID (2,5K blocks, F=30), `Order(ORDERID, CustId, TotalAmount, Date)` has 250K tuples in a primary sequential storage with tuples ordered by Date, with 3K blocks. Table Order has a B+ index on CustId with 300 leaf nodes, 3 levels, and a B+ index on Date with 400 leaf nodes, 3 levels. 10% of the orders are in the specified interval and these orders involve 500 customers. Describe an efficient plan for the two queries below. Write the complete formulas of their costs.

(Query 1)

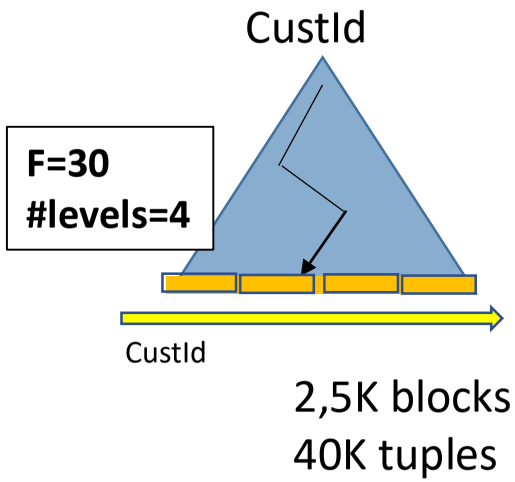
```
select Name, Email, Phone
from Customer C
where exists (select *
              from Order O
              where C.CustId = O.CustId and Date between "21/07/2021" and "20/07/2022")
```

(Query 2) – **skip this query if you were granted a 30% reduction.**

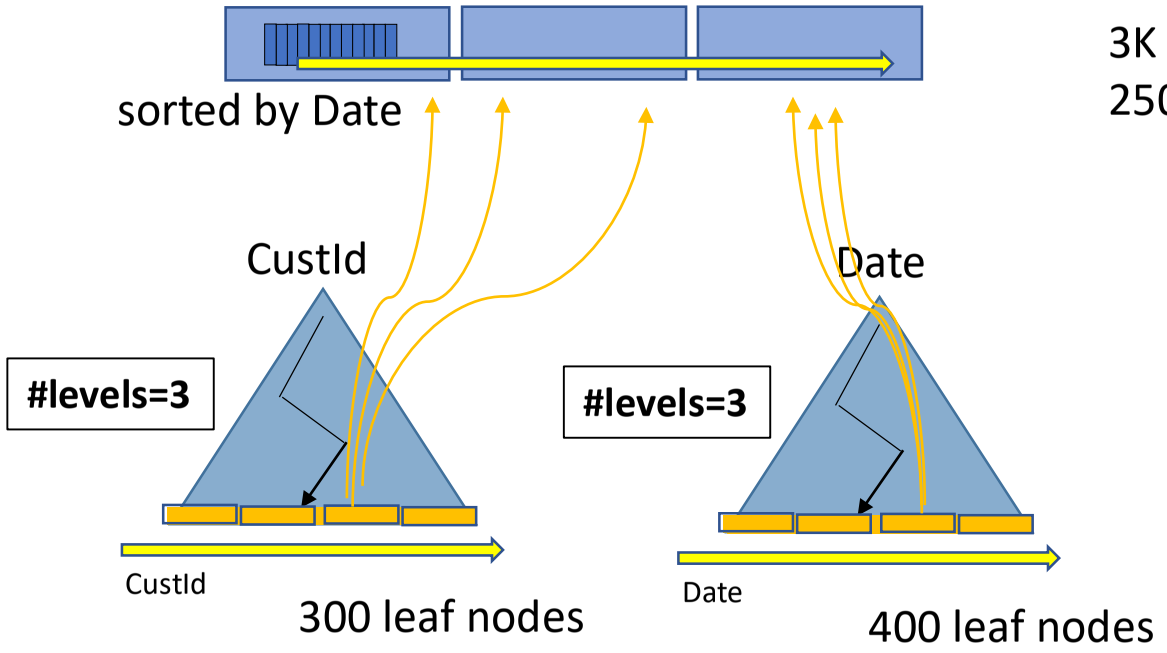
```
select Name, Email, Phone
from Customer C
where not exists (select *
                  from Order O
                  where C.CustId = O.CustId and Date between "21/07/2021" and "20/07/2022")
```

Solution. See separate file

CUSTOMER(CustId, Name, Email, Phone,…)



ORDER(OrderId, CustId, TotalAmount, Date)



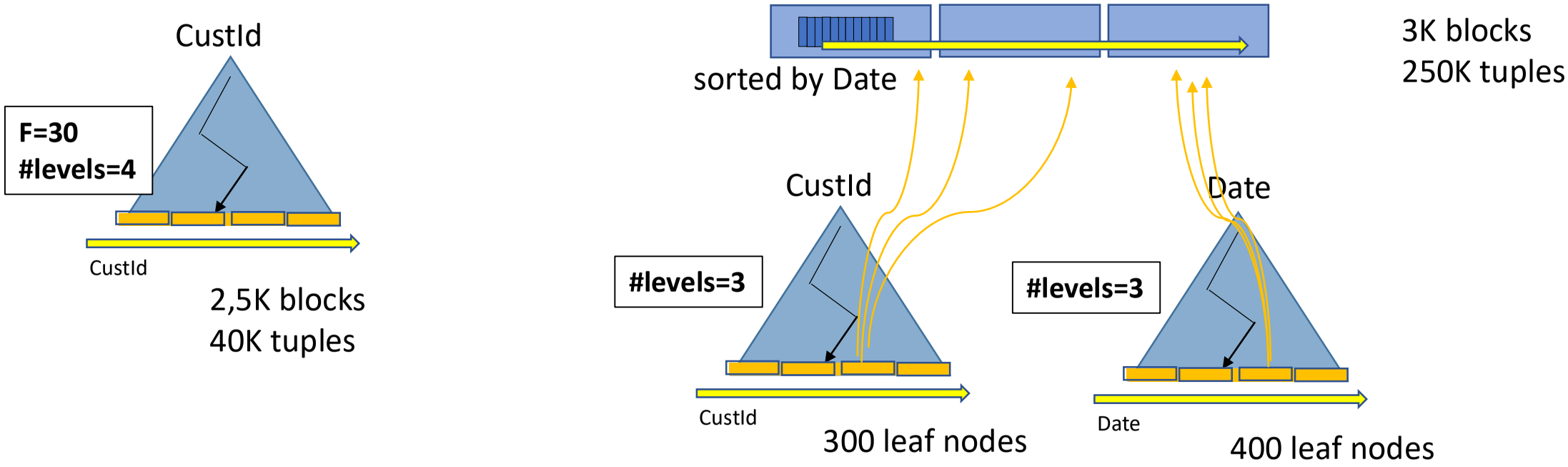
1. Do a join on CustomerId on the B+(CustId)
   Cost = 3 + 2500 + 2 + 300

2. Follow all the 250K pointers to check the date
   Cost = (3 + 2500 + 2 + 300) + 250K = 252,8K

This plan can answer both query 1 and query 2

CUSTOMER(CustId, Name, Email, Phone,…)

ORDER(OrderId, CustId, TotalAmount, Date)



1 Find "21/07/2021" in the B+(Date) and follow the (first) link

2 Read all the consecutive blocks in ORDER primary sequential structure to read all the orders in the specified interval
→ Read 10% of 3000 blocks
Get (and cache) the 500 CustId of the customers involved

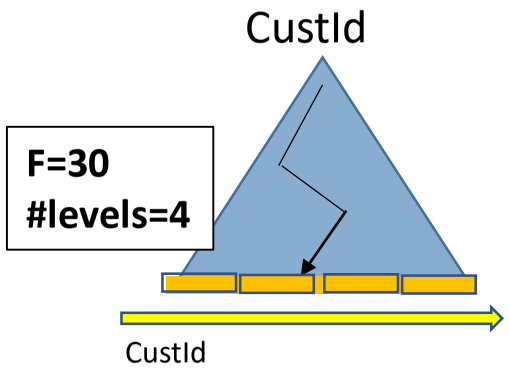
3 For each of the 500 customers get the full tuple, by accessing 4 levels

This plan can answer only query 1

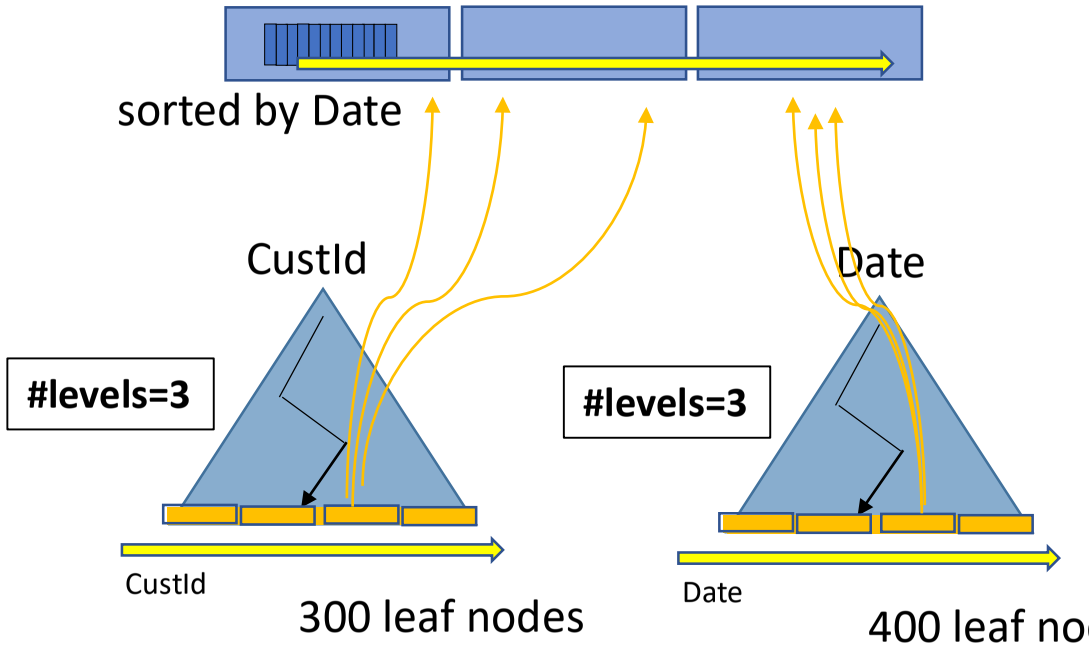
Total cost = 3 (B+(date)) + 10%*3000 (Order blocks) + 500*4 (Customers)= 2303 I/O accesses

CUSTOMER(CustId, Name, Email, Phone,…)

ORDER(OrderId, CustId, TotalAmount, Date)



2,5K blocks
40K tuples



3K blocks
250K tuples

300 leaf nodes

400 leaf nodes

1 Find "21/07/2021" in the B+(Date) and follow the (first) link

2 Read all the consecutive blocks in ORDER primary sequential structure to read all the orders in the specified interval
→ Read 10% of 3000 blocks
Get (and cache) the 500 CustId of the customers involved

3 Read all the 2500 leaf nodes and compare the customers with the cached customers

This plan can answer Query 1 and Query 2

Total cost = 3 (B+(date)) + 10%*3000 (Order blocks) + 3 + 2500 (Customers)= 2806 I/O accesses

C. Triggers (9 points)

A logistics company manages shipments via trucks. Each driver owns one or more trucks and has a number of colleagues who can drive his/her trucks. A truck makes zero or more trips. A trip specifies that a given truck must travel from an origin to a destination on a certain date. When the driver completes a trip, a report is created with the number of hours travelled. Given the following logical schema (keys in uppercase letters):

```
TRUCK(PLATE, status, ownerId) -- status: available (default), maintenance
DRIVER(ID, hoursTravelled, status) -- status: authorized (default), non-authorized
                                   -- hoursTravelled defaults to 0
TRIP(TRIPID, date, origin, destination, truckPlate, driverId)
TRIPREPORT(TRIPID, hoursTravelled)
ALTERNATE(DRIVERID, ALTERNATEDRIVERID) -- DRIVERID is the owner
```

Suppose that when a new trip is created, the initial values for the `truckPlate` and `driverId` are set to NULL. Design a set of triggers that implement the following behaviours

- When a truck is assigned to a trip, if the status of the truck is `maintenance` the operation is prevented from proceeding and the trigger raises an exception with text `Truck under maintenance`. After the assignment of an available truck to a trip, a trigger tries to automatically assign a suitable driver, giving priority to the truck's owner. He/she can be assigned only if his/her status is `authorized` and he/she is available for that date. Otherwise, an alternate driver is searched: the one with the least cumulative travel hours among those with `authorized` status available for that date is chosen. If a suitable driver is not found, the trigger raises an exception with text `No driver available`.
- When a trip report is created, the total number of hours travelled by the driver must be updated. If the hours s/he has travelled exceeds 1000, his/her status is set to `non-authorized` and his/her assignments to the trips posterior to the reported one are set to NULL. **Skip this part if you were granted a 30% reduction.**

An example of the syntax for signalling a specific error is:

```
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = "some error occurred";
```

Solution.

1.
```
CREATE TRIGGER CheckMaintenance
BEFORE UPDATE OF truckPlate ON TRIP
FOR EACH ROW
BEGIN
    DECLARE truck_status VARCHAR(20);

    SELECT status INTO truck_status
    FROM TRUCK
    WHERE PLATE = NEW.truckPlate;

    IF truck_status = 'maintenance' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Truck under maintenance';
    END;
END;
```

2.
```
CREATE TRIGGER AssignDriver
AFTER UPDATE OF truckPlate ON TRIP
FOR EACH ROW
BEGIN
```

```
    DECLARE owner_id INT;
    DECLARE suitable_driver_id INT;

    SELECT ownerId INTO owner_id
    FROM TRUCK
    WHERE PLATE = NEW.truckPlate;

    -- Try to assign the owner as driver if authorized and available
    SELECT ID INTO suitable_driver_id
    FROM DRIVER
    WHERE ID = owner_id AND status = 'authorized'
    AND NOT EXISTS (
        SELECT *
        FROM TRIP T
        WHERE T.driverId = owner_id
        AND T.date = NEW.date
    )

    -- If the owner is not available or authorized, find an alternate driver
    IF suitable_driver_id IS NULL THEN
        SELECT A.ALTERNATEDRIVERID INTO suitable_driver_id
        FROM ALTERNATE A
        JOIN DRIVER D ON A.ALTERNATEDRIVERID = D.ID
        WHERE A.DRIVERID = owner_id
        AND D.status = 'authorized'
        AND NOT EXISTS (
            SELECT *
            FROM TRIP T
            WHERE T.driverId = A.ALTERNATEDRIVERID
            AND T.date = NEW.date
        )
        ORDER BY D.hoursTravelled ASC
        LIMIT 1;
    END IF;

    IF suitable_driver_id IS NOT NULL THEN
        UPDATE TRIP
        SET driverId = suitable_driver_id
        WHERE NEW.TRIPID = TRIPID;
    ELSE
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'No driver available';
        END IF;
    END IF;
END;
```

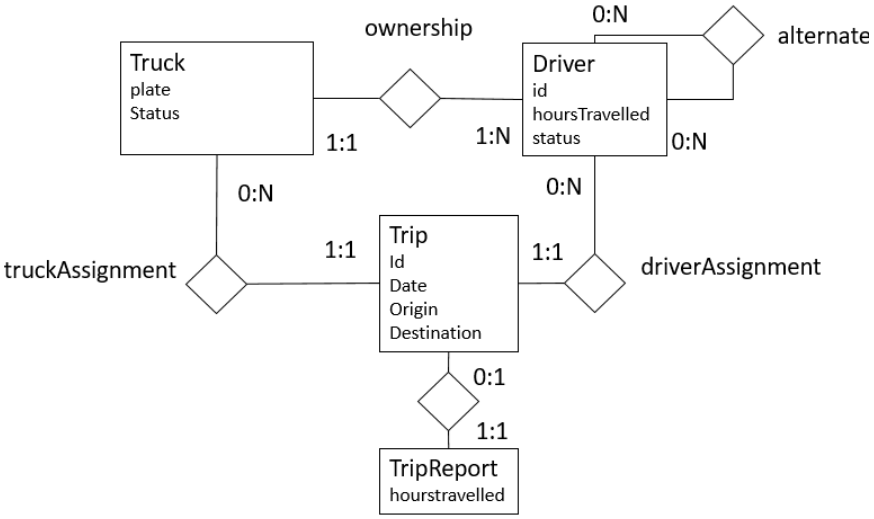
3.
```
CREATE TRIGGER UpdateDriverStatus
AFTER INSERT ON TRIPREPORT
FOR EACH ROW
BEGIN
   UPDATE DRIVER
   SET hoursTravelled += NEW.hoursTravelled
   WHERE ID = (SELECT driverId FROM TRIP WHERE TRIPID = NEW.TRIPID);
END;


CREATE TRIGGER UpdateDriverStatus
AFTER UPDATE OF hoursTravelled ON DRIVER
FOR EACH ROW
WHEN NEW.hoursTravelled > 1000
```

```
BEGIN
   UPDATE DRIVER
   SET status = 'non-authorized'
   WHERE ID = NEW.ID;

   UPDATE TRIP
   SET driverId = NULL
   WHERE driverId = (SELECT driverId FROM TRIP WHERE TRIPID = NEW.TRIPID) AND date > NOW();
END;
```

D. JPA (7 points)
In the web application of the logistic company, the user can access the list of available drivers, select one driver and access his/her alternate drivers and trips s/he has made, in descending order of date; access the list of trucks and see the trips associated with each truck, in descending order of date; access the list of trips in descending order of date and check the truck and driver associated to each trip. Trucks are in the order of thousands, drivers are in the order of tens of thousands, trips are in the order of millions.



Show the JPA entities (with all their attributes) that map the domain objects `TRUCK DRIVER` and `TRIP` and the respective relationships (`ownership, alternate, truckAssignment, driverAssignment`) of the conceptual model, taking into account the above-mentioned access paths of the application and data cardinalities. When designing the annotations for the relationships, specify the owner side of the relationship, the mapped-by attribute, the fetch policy and the cascading policies you consider more appropriate to support the access required by the web application. Comment on the design choices you have made.
Solution.

- Relationship **Ownership**: no direction is navigated according to the requirements. In any case, owner would be *Truck* and eager fetch and cascading of remove would apply to both sides.
- Relationship **truckAssignment**: both directions are requested. Owner is *Trip*. Cascading may be applied from *Truck* to *Trip*. Fetch type is default (lazy for truck to trips and eager for trip to the unique truck)
- Relationship **driverAssignment**: both directions are requested. Owner is *Trip*. Cascading may be applied from *Driver* to *Trip*. Fetch type is default (lazy for driver to trips and eager for trip to the unique driver)
- Relationship **Alternate**: only one direction needed (from a driver to his/her alternates). Cascading is not necessary. Owner undefined (many to many). Fetch type can be eager (alternative drivers are few).

```
@Entity
public class Truck implements Serializable {
private static final long serialVersionUID = 1L;
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
        private int id;
        private String plate;
        private String status;
```

```
@OneToMany(mappedBy = "truck", fetch = FetchType.LAZY, cascade = CascadeType.REMOVE)
private List<Trip> tripsOfTruck;
 . . .
}

@Entity
public class Driver implements Serializable {
private static final long serialVersionUID = 1L;
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
        private int id;
        private int hoursTravelled;
        private String status;
@OneToMany(mappedBy = "driver", fetch =  FetchType.LAZY, cascade = CascadeType.REMOVE))
private List<Trip> tripsOfDriver;

@ManyToMany(fetch = FetchType.EAGER) // no mappedBy  because  inverse is not implemented
@JoinTable(
name="alternates",
       joinColumns={@JoinColumn(name="driverid")},
       inverseJoinColumns = {@JoinColumn(name="alternatedriverid")}
private List<Driver> alternateDrivers;

 . . .
}

@Entity
public class Trip implements Serializable {
private static final long serialVersionUID = 1L;
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
        private int id;
@Temporal(TemporalType.DATE)
        private Date tripdate;
        private String origin;
        private String destination;

@OrderBy("tripdate DESC")

@ManyToOne @JoinColumn(name="truckplate")
private Truck truck;  //owner

@ManyToOne @JoinColumn(name="driverid")
private Driver driver;  //owner
 . . .
}
```